SHENZHEN UNIVERSITY

COURSE

Data Analytics in Marketing
Shenzhen University

Chapter 15 — Big Data in Marketing
Measurement, choice, response and customer value

Lecturer: Xieer Dai (David)

Shenzhen University · Week 14 · Thursday, periods 11–12

Course 0205360001 · Data Analytics in Marketing

Shenzhen University | Office hours by appointment | xieer.dai@szu.edu.cn



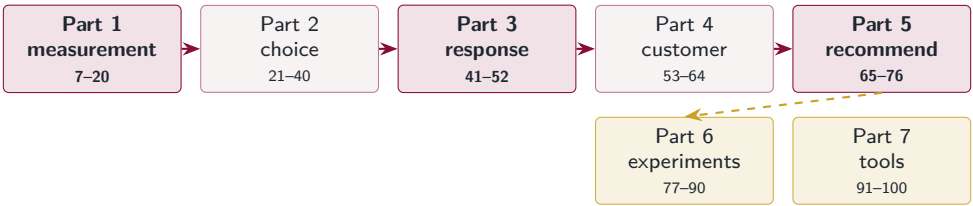
Learning Objectives

By the end of this chapter you should be able to:

1. design a trustworthy scale questionnaire and test its reliability and validity;
2. estimate attribute utilities and importance weights with conjoint analysis;
3. turn utilities into market share with a logit choice model;
4. estimate hedonic prices and response curves, and compute price elasticity and the profit-maximising price;
5. build a funnel, an attribution split, a churn model and a CLV, and validate the effect with an A/B test and a power calculation;
6. explain what the new privacy rules do to marketing data, and write a compliant measurement plan.

Source: Course outline, session 15: master marketing data collection and experimental design; support decisions with conjoint analysis, recommenders and customer-value models.

Roadmap (100 frames)



Main line: measurement → choice → response → customer → recommendation and price → experimental check

10 code frames · 14 sample frames; every term carries its definition

From Toolkit to Marketing

Chapter	What it gave you	What it becomes here
Ch. 2 / Ch. 3	sampling, questionnaires, design	scales and A/B tests (frames 11, 84)
Ch. 5 (supervised)	classification and regression	response models, churn, elasticity (frames 46, 78, 71)
Ch. 5 (unsupervised)	clustering and dimension reduction	segments and RFM scoring (frame 54)
Ch. 5 (deep nets)	neural networks	recommendation and content (frames 68, 89)
Ch. 4	charts and visual grammar	funnel and attribution views (frames 56, 60)

Marketing is not a new method; it is applying existing methods to data about people. Every method here ships with a number you can compute, so you can carry it straight into a project.

Source: The mapping is this chapter's own organisation. Causal inference is outside this course's chapter sequence and is only cited as a method.

Deliverables

1) Scales and design

An item table, a reliability and validity check, and a conjoint attribute–level table with its orthogonal design.

2) 10 programs

Reliability, conjoint, logit, hedonic, funnel and attribution, recommendation, elasticity, churn, CLV, A/B.

3) Glossary

85 entries in the Chinese edition, grouped into six topics; the English terms are kept throughout this deck.

Every number is reproducible: `website/code/lecture15/`, seed 20261216; all data are synthetic (600 questionnaires, 800 choices, 1,200 homes, 6,000 sessions, a 500×60 rating matrix, 5,000 users per arm).

Source: Programs and data ship with the course repository.

Map of the Lecture

Part	Frames	Key question
Marketing data and measurement	7–20	Is the measurement any good? How should a questionnaire be built?
Conjoint analysis and choice	21–40	What is an attribute worth? Which option will people pick?
Hedonic prices and response	41–52	What sets the price? How much spend pays off?
Consumer data and attribution	53–64	How does the customer move? Who gets the credit?
Recommendation, price, promotion	65–76	What to recommend? What to charge? How deep to discount?
Churn, CLV and experiments	77–90	Who leaves? What are they worth? Did it work?
Tools and glossary	91–100	Which tools? How do I remember the terms?

How to read it: every method walks the same path — question → design → estimate → sample figure → how to use it.

Source: The map follows this chapter's own structure.

Four Questions of Marketing Analytics

Question	Typical wording	Method	Frames
Who buys	What do customers look like, will they leave	segments, churn model	54, 78
What they buy	Which configuration do they prefer	conjoint, recommendation	21, 65
Why	Did the advertising work	attribution, A/B, incrementality	58, 84
What it is worth	What is this customer worth to us	CLV, price elasticity	80, 71

Decide which of the four you are asking before you decide what data to collect — the question determines the data structure. This is the first fork in marketing analytics.

Source: The four questions follow the standard split in marketing analytics: description, preference, effect, value.

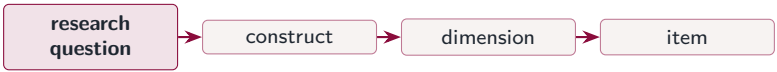
Five Data Sources

Source	Typical content	Main bias	Cost
Survey	attitudes, intentions, demographics	self-report and sample bias	medium
Transaction / CRM	purchases, amount, frequency	you only see buyers	low
Clickstream	impressions, clicks, paths	missing events, bots	medium
Social media	content, engagement, word of mouth	platform definitions, fake accounts	medium
Experiment	randomised control	spillover, novelty effect	high

The best combination is attitudes (survey) + behaviour (transactions, clicks) + causality (experiment): the survey says *why*, behaviour says *what happened*, the experiment says *whether it truly worked*.

Source: The five sources and their biases follow marketing research practice; the combination is this chapter's recommendation.

Questionnaire Design (1): From Question to Item



Example: brand trust

Construct = brand trust; dimensions = ability, benevolence, integrity; items = "this brand will not let me down", "I trust its quality".

How many items per construct

Usually 3–5: fewer is unstable, more exhausts the respondent. This chapter uses 4 (the reliability on frame 14 rests on them).

The most common mistake: copying items from the literature without saying which dimension of which construct each one measures.

Source: The construct–dimension–item operationalisation follows Churchill's (1979) scale-development paradigm.

Questionnaire Design (2): Item Types and Wording Traps

Four item types

- Likert: degree of agreement on 1–5;
- Ranking: order the alternatives;
- Choice: pick one of several profiles (frame 30);
- Open text: a paragraph of free text.

Four wording traps

1. Double-barrelled: "is service good *and* price low?";
2. Leading: "you agree it is unreasonably expensive, right?";
3. Social desirability: under-report smoking, over-report reading;
4. No default: without "don't know" you force an answer.

Self-check: read every item to a colleague and ask "how would you answer?" — if two people read it differently, the item has to change.

Source: The four item types and four traps follow questionnaire design practice (Dillman's tailored design).

Sampling: Probability and Non-probability

Scheme	How it works	Can you infer to the population
Simple random	equal-probability draw from a list	yes
Stratified	draw within strata, then pool	yes, and more precise
Cluster	draw clusters, census inside them	yes, at lower cost
Quota	fill the quotas proportionally	no (non-probability)
Convenience	whoever is easy to reach	no

Sample size for a proportion: $n \approx 1.96^2 p(1-p)/e^2$ — at $p = 0.5$ and a 3% margin you need about 1,067 responses. In marketing projects the real problem is usually that the sample is too young and too urban, not that it is too small.

Source: The schemes and the sample-size formula follow standard sampling theory; the observation on marketing samples is this chapter's addition.

Modes of Administration and Mode Effects

Four modes

Face to face (deep but expensive), telephone (fast but short), web (cheap but skewed), mixed (complementary but hard to compare).

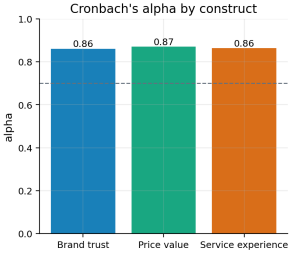
Mode effects

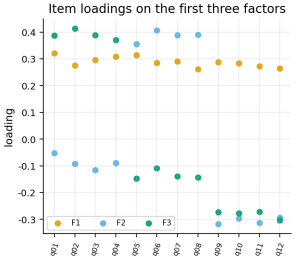
On the same question, a face-to-face interview is pushed by social desirability towards fewer negative answers; a web survey gets more honesty but a younger sample.

So changing the mode is changing the ruler — if the mode changed between two waves, the trend comparison has to be discounted. Say so in the method note.

Source: Mode effects follow the survey-methodology literature; the practical warning is this chapter's addition.

[Sample + Visualisation] The Skeleton of a Marketing Survey







Left: Cronbach's α for the three constructs (brand trust, price value, service experience), all above the 0.7 threshold. Middle: loadings of the 12 items on the first three factors — each construct's items cluster together. Right: the correlation matrix of the three construct scores, with very low cross-construct correlations (discriminant validity holds).

Program `115_01_survey_reliability.py` · sample `115_survey.csv` (600 respondents, 12 items)

Reliability: Does the Same Thing Measure the Same Twice

Three kinds of reliability

1. internal consistency: Cronbach's α;
2. test–retest: measure again two weeks later;
3. split-half: odd and even items scored separately.

The formula and its trap

$$\alpha = \frac{k}{k-1}\left(1 - \frac{\sum \sigma_i^2}{\sigma_T^2}\right).$$

α rises with the number of items: twenty bad items can still reach 0.8.

Measured here: α between 0.72 and 0.81 across the three constructs, 0.761 for the whole scale. Reading rule: > 0.7 usable, > 0.8 good — but always report the item count and the construct definition.

Source: The formula and the reading rule follow Cronbach (1951) and common practice.

Validity: Are We Measuring the Right Thing

Type	The question it asks	How to check
Content	do the items cover every dimension	expert review, pre-test
Construct	are we really measuring the abstract construct	factor analysis (frame 16)
Convergent	do two scales of the same construct agree	correlation with an existing scale
Discriminant	do different constructs stay apart	correlation below $\sqrt{\text{AVE}}$
Criterion	does it predict what it should predict	correlation with purchase behaviour

One sentence: reliability is necessary, validity is what you actually need — a high α says the measurement is *stable*, not that it is *right*.

Source: The five types follow Campbell & Fiske (1959) and scale-development practice.

[Code] Reliability, Exploratory Factor Analysis and Discriminant Validity

```python
def cronbach_alpha(frame):                          # reliability of a block of items
    k = frame.shape[1]                              # number of items
    var_items = frame.var(axis=0, ddof=1).sum()      # sum of item variances
    var_total = frame.sum(axis=1).var(ddof=1)        # variance of the total score
    return float(k / (k - 1) * (1 - var_items / var_total)) # the standard formula
rows = []                                           # one row per construct
for name, cols in blocks.items():                   # loop over the constructs
    alpha = cronbach_alpha(survey[cols])            # reliability
    rows.append({"construct": name, "items": len(cols), "alpha": round(alpha, 3), # the row
                 "mean": round(mean, 2)})           # more columns
X = StandardScaler().fit_transform(survey[items])   # standardise the twelve items
pca = PCA(n_components=4).fit(X)                    # four components
loadings = pd.DataFrame(pca.components_.T, index=items, # item loadings
                        columns=["F1", "F2", "F3", "F4"]) # four factor columns
corr = scores.corr()                                # construct correlations
sqrt_ave = float(np.sqrt(avg_var))                  # square root of the average variance
```

Measured: $\alpha = 0.72$–0.81, 0.761 for the whole scale; the four components explain 0.61 of the variance (the first two dominate); cross-construct correlations run 0.06–0.09, far below $\sqrt{\text{AVE}}$, so discriminant validity holds. Full program 115_01.

Measurement Error and Common Method Bias

Two kinds of error

Random error makes estimates noisy (it averages out); systematic error makes them biased (it does not — and that is the dangerous one).

Common method bias

One questionnaire, one respondent, one moment for every variable — so the correlation between variables may be produced by the method.

Three remedies: separate in time (pre and post), separate in source (self-report plus an objective record), test statistically (Harman's single factor: if the first factor explains less than half, you are probably fine).

Source: Common method bias and Harman's test follow Podsakoff et al. (2003).

Non-response and Weighting

Three kinds of missing response

- unit non-response: the whole questionnaire is not returned;
- item non-response: one question is skipped;
- coverage error: some group is not on the list at all.

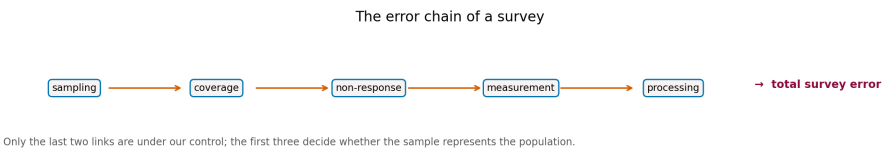
Two treatments

Weighting: reweight the sample to the population structure; imputation: fill gaps from other variables (and document the method).

To judge the bias, compare respondents and non-respondents on known variables (age, city, past spend). This chapter has no real non-respondents, so that step has to be done on your own project data.

Source: The typology and weighting follow survey methodology; the judgement step is this chapter's addition.

[Sample + Visualisation] The Error Chain of a Survey



From sampling to processing, every link adds error; together they make up total survey error. This chapter can only change the last two links (measurement and processing); the first three decide whether the sample represents the population — which is why the sampling design on frame 11 and the non-response analysis on frame 18 matter more than the choice of model.

Program `115_01_survey_reliability.py` · figure drawn for this chapter

Summary: Measurement Sets the Ceiling

Three sentences

Operationalise the construct into items; reliability asks whether the measurement is consistent, validity whether it is right; every link of the error chain has to be reported.

What it means for a project

If the questionnaire has three items and no reliability check, then every regression coefficient downstream is not worth interpreting.



Next part: turn *attributes* into *value*, and then *value* into *choice*.

Source: Summary prepared for this chapter.

Conjoint Analysis: Value the Bundle, Not the Attribute

Ask about attributes directly

"Does range matter?" — everyone says yes. But when people actually buy, range has to be paid for with price.

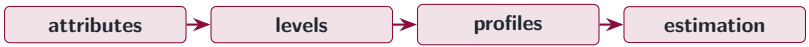
What conjoint does instead

Give the respondent a whole bundle (650 km, RMB 230k, 20-minute charging) and ask for a rating or a choice; the trade-off then appears in the data by itself.

So the core of conjoint is the trade-off: every utility point is a marginal value *holding the other attributes fixed*. Paul Green brought the method from mathematical psychology into marketing in the 1970s.

Source: The origin of conjoint follows Green & Rao (1971); the trade-off framing is this chapter's.

Conjoint in Four Steps



The decision at each step

4–6 attributes (more and respondents lose track), 2–5 levels each, control of the profile count (full factorial versus orthogonal), estimation (OLS, logit, or hierarchical Bayes).

What this chapter does

4 attributes $\times$ 3 levels $=$ a full factorial of 81 profiles; a balanced design keeps 12 of them (frame 25 shows every level appearing equally often).

Source: The four steps follow Green & Srinivasan (1978, 1990); the parameters here are this chapter's.

Choosing Attributes and Levels

Three conditions

1. Actionable: the firm can really change it (price, configuration);
2. Tradeable: respondents are willing to swap (range against price);
3. Not collinear: attributes must not be tied together by construction.

Setting the levels

Cover the real market range: too low is uncompetitive, too high nobody buys. Price levels in particular must sit inside the competitors' band.

Counter-example: put "brand" and "price" in the same design when premium brands are always expensive — the two are highly collinear and the estimated utilities cannot be separated. Either change the design or use a constrained model.

Source: The three conditions and the collinearity problem follow conjoint practice; the counter-example is this chapter's.

Design Types: Full Factorial, Orthogonal, D-optimal

Design	Profiles	Advantage	Cost
Full factorial	$3^4 = 81$	interactions are estimable	respondents cannot finish it
Orthogonal	9–18	main effects estimable, balanced	interactions usually not
Random balanced	12–24	simple, no collinearity	balance has to be checked
D-optimal	model-dependent	most efficient parameters	needs priors and an algorithm

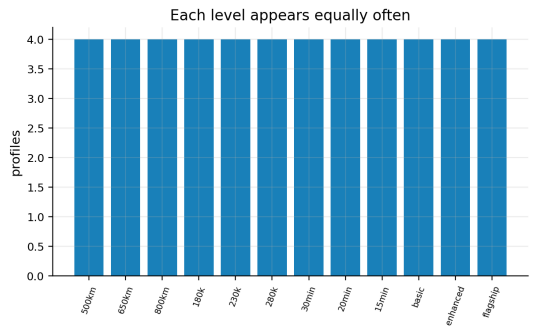
This chapter uses a random balanced design: each attribute is shuffled independently and every level appears the same number of times. The gain is that it is free of collinearity; the cost is that profiles differ across respondents, so individual importance weights are not directly comparable.

Source: The design types follow the conjoint literature (Addelman's 1962 orthogonal main-effect designs); the choice here is in 115_02.

[Sample + Visualisation] Attributes, Levels and the Balanced Design

Balanced design (12 profiles, full factorial 81)

profile	range	price	charging	assist
P01	800km	180k	15min	flagship
P02	650km	280k	20min	enhanced
P03	800km	280k	30min	enhanced
P04	500km	180k	30min	enhanced
P05	650km	230k	30min	basic
P06	500km	230k	30min	basic
P07	500km	230k	15min	flagship
P08	500km	180k	15min	enhanced



Left: the attribute combinations of the 12 profiles (range 500/650/800 km, price 180k/230k/280k, charging 30/20/15 minutes, driver assist basic/enhanced/flagship); right: every level appears exactly four times — that is what "balanced" means, and it is what makes the utilities estimable and the importance weights comparable.

Program `115_02_conjoint.py` · design table `115_conjoint_design.csv` (full factorial 81 → balanced design 12)

Estimation: Effect Coding and Part-worths

Why effect coding

One dummy per level, coded $+1$ / -1 instead of $1/0$: the utilities of a level set then sum to zero, so each one reads directly as "above or below the average".

How importance is computed

Take the utility range of each attribute (max minus min), then divide by the sum of all ranges:

$$\text{imp}_j = \frac{\max u_j - \min u_j}{\sum_k (\max u_k - \min u_k)}.$$

Measured here ($R^2 = 0.672$): price 35.8%, range 29.3%, driver assist 18.7%, charging 16.2%, against the true weights used to generate the data (36.7 / 29.1 / 17.7 / 16.5) — close enough to show that the design is estimable.

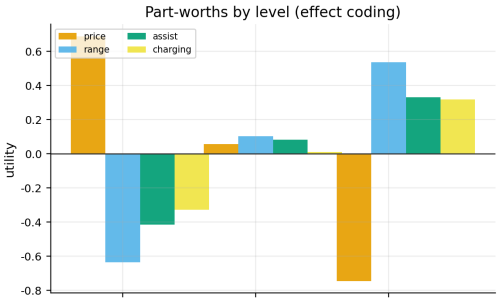
Source: Effect coding and the importance formula follow standard conjoint practice; the numbers come from 115_02.

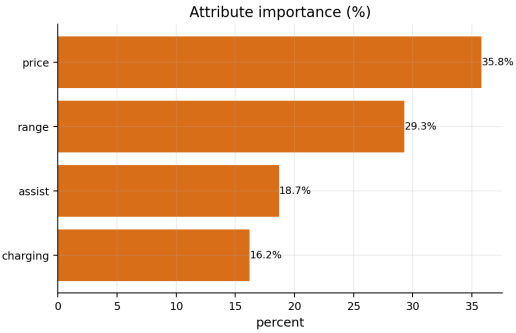
[Code] Conjoint: Design, Estimation and Importance

```
design = make_conjoint_profiles(12)                    # twelve balanced profiles
full = np.prod([len(v) for v in ATTRS.values()])       # the full factorial size
responses = make_conjoint_responses(design, 300)       # 300 respondents x 12 profiles
merged = responses.merge(design, on="profile")         # attach the attribute levels
for attr in ATTRS:                                     # loop over the attributes
    levels = ATTRS[attr]                               # its levels
    for lv in levels[:-1]:                             # all but the last level
        col = "d_%s_%s" % (attr, lv)                   # dummy column name
        merged[col] = (merged[attr] == lv).astype(float) # 1 for this level
        merged[col] = merged[col] - (merged[attr] == levels[-1]) # effect coding: -1 on the reference
formula = "rating ~ " + " + ".join([c for c in merged.columns if c.startswith("d_")]) # the model
model = smf.ols(formula, data=merged).fit()            # OLS on the stacked data
for attr in ATTRS:                                     # loop over attributes
    spread = max(estimated.values()) - min(estimated.values()) # the utility range
    for lv, u in estimated.items():                    # write one row per level
        rows.append({"attribute": attr, "level": lv, "part_worth": round(u, 3), # the record
                     "range": round(spread, 3)})       # attribute range
```

Measured: $R^2 = 0.672$; importance price 35.8% / range 29.3% / assist 18.7% / charging 16.2%. Full program `115_02_conjoint.py` (English comment on every line).

[Sample + Visualisation] What Utilities and Importance Look Like





Left: the part-worth of every level — moving the price from 180k to 280k costs 1.43 utility points. Right: the importance ranking. This is the chart a pricing or configuration decision is made on: a 50k price cut from 230k adds about 0.63 utility, which is more than the 0.43 that going from 650 km to 800 km of range is worth.

Program `115_02_conjoint.py` · utilities `115_conjoint_utilities.csv` · importance `115_conjoint_importance.csv`

Individual-level Preferences, and the Pros and Cons

Individual level

Estimate once per person (here 300 people $\times$ 12 profiles) and you get preference heterogeneity — the input to segmentation and personalised pricing. The steadier route is hierarchical Bayes, which shrinks each individual estimate towards the sample.

Pros and cons

Pros: close to a real trade-off, utilities are interpretable, the model can be used to simulate. **Cons**: a long questionnaire, attributes are assumed independent, and stated ratings still differ from real purchases.

Empirically, conjoint shares correlate with actual market shares at about 0.6–0.8: it ranks well, but do not use it as the market share itself.

Source: Pros and cons follow the module reading; the 0.6–0.8 correlation is the usual range in conjoint validation studies.

Why Choice Models Rather Than Ratings

The problem with ratings

Some respondents use only 7 (centring bias), others use only 1 and 7; the *distance* between two scores is not comparable across people.

The advantage of choice

A choice is close to real behaviour; no scale assumption is needed; and it produces shares directly.

The price: a choice carries less information than a rating (one choice is one bit), so you need more respondents or more choice occasions. That is why a discrete choice experiment usually runs on 300–1,000 people.

Source: The comparison of ratings and choices follows Louviere et al. (2000); the sample-size guidance is this chapter's.

Three Kinds of Discrete-Choice Data

Type	Unit of observation	What it can estimate
Individual	one choice by one person	preference heterogeneity, individual utilities
Aggregate	market share, regional sales	average utility (ecological regression)
Panel	the same people choosing repeatedly	state dependence, learning

This chapter uses individual-level data (800 choice occasions), because it supports both the average preference and a heterogeneity analysis.

Source: The typology follows the module reading and Train (2009).

Random Utility and the Logit Derivation

The setup

Utility of alternative j is $U_j = V_j + \varepsilon_j$, and the consumer picks the largest:

$$P_j = \Pr(U_j \geq U_k \ \forall k).$$

Gumbel errors $\rightarrow$ logit

If ε is i.i.d. Gumbel, the probability has a closed form:

$$P_j = \frac{e^{V_j}}{\sum_k e^{V_k}}.$$

That is the conditional logit.

If the errors are normal rather than Gumbel there is no closed form and you need probit (numerical integration or simulation). The estimated coefficients are close; the difference shows up mainly in the substitution pattern.

Source: The random-utility derivation follows McFadden (1974); the probit comparison follows Train (2009).

[Code] Conditional Logit: Maximum Likelihood and Marginal Effects

```
def neg_loglik(beta):                                  # negative log-likelihood
    v = X @ beta                                       # deterministic utilities
    total = 0.0                                        # accumulator
    for g in np.unique(groups):                        # loop over choice sets
        mask = groups == g                             # the rows of this set
        vg = v[mask]                                   # utilities in the set
        vg = vg - vg.max()                             # stabilise with the max trick
        denom = np.log(np.exp(vg).sum())               # log-sum-exp denominator
        total += (vg[y[mask].astype(bool)].sum() - denom) # log probability of the choice
    return -total                                      # maximise, so negate
result = minimize(neg_loglik, np.zeros(X.shape[1]), method="BFGS") # start from zeros
beta = result.x                                        # the estimates
H = hessian(neg_loglik, beta)                          # Hessian at the estimates
se = np.sqrt(np.diag(np.linalg.inv(H)))                # standard errors
table = pd.DataFrame({"term": names, "coef": np.round(beta, 4), "se": np.round(se, 4), # the table
                      "z": np.round(z, 2), "p": np.round(pvals, 4)}) # inference columns
```

Measured: price -0.246, range $+0.0046$, charging -0.092 (per minute), brand A $+0.17$, B $+0.63$; McFadden pseudo $R^2 = 0.302$. The true generating values (-0.22 / 0.004 / -0.09 / 0.35) are recovered closely. Full program `115_03_choice_logit.py`.

Binary Choice: Logit and Probit

The form

$$\Pr(y=1) = \Lambda(x^\top \beta) = \frac{1}{1+e^{-x^\top \beta}},$$

while probit uses $\Phi(x^\top \beta)$.

Reading a coefficient

You cannot read it as a marginal effect — in a logit the marginal effect moves with x. Report the average marginal effect or the odds ratio e^{β}.

Example: $\beta = 0.5$ gives an odds ratio of $e^{0.5} = 1.65$ — "one more unit multiplies the odds by 1.65", not the probability.

Source: The logit/probit reading follows standard econometrics texts (Wooldridge).

Multinomial Logit and the Independence of Irrelevant Alternatives

What IIA says

Add a new alternative and it takes share from every existing one in proportion; the ratio of any two shares stays the same.

The red bus / blue bus paradox

Car and blue bus at 50% each; add a red bus identical to the blue one and the conditional logit says 33% each — but in reality the red bus steals from the blue one.

Measured here: adding a new alternative D that is close to C leaves the A/B ratio at 4.307 before and 4.307 after — exactly unchanged, which is IIA. Real data usually rejects it, which is why nested or mixed logit exists (frame 36).

Source: IIA and the red bus / blue bus paradox follow McFadden (1974) and Train (2009); the numbers come from 115_03.

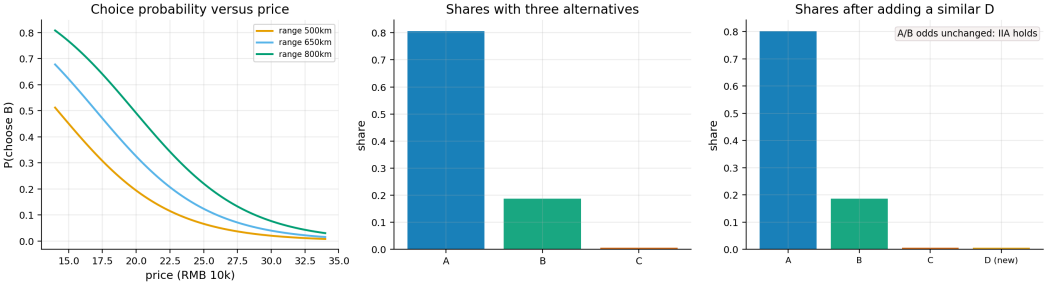
Nested and Mixed Logit: When They Are Required

Model	Assumption relaxed	When to use it
Conditional logit	none: no correlation, fixed coefficients	the baseline
Nested logit	correlation within a nest, independence across nests	there is a natural "category first, model second"
Mixed logit	random coefficients (heterogeneity)	preferences differ widely, individual utilities needed
Latent class logit	discrete preference segments	you want segments, not a continuous distribution

How to choose: look first at the substitution pattern (who takes whose share), then at the distribution of preferences (homogeneous $\rightarrow$ fixed coefficients, heterogeneous $\rightarrow$ random coefficients). This chapter covers the logic only; estimation belongs to a later course.

Source: The model family and the choice rule follow Train (2009) and Greene's econometrics text.

[Sample + Visualisation] Choice Probabilities and the IIA Demo



Left: how the probability of choosing B falls as the price runs from 140k to 340k — the longer the range, the flatter the curve, so the less price-sensitive the choice. Middle: the shares of three alternatives. Right: after adding a D that is close to C, the A/B ratio barely moves (IIA).

Program `l15_03_choice_logit.py` · data `l15_choice_long.csv` (800 choice occasions)

Example: How an EV Policy Shifts Petrol-Car Demand

The question

Subsidies and licence privileges make electric cars cheaper. How much petrol-car demand disappears — and which models lose the most?

How a choice model answers it

Put petrol, plug-in hybrid and battery-electric cars in the same choice set; the subsidy changes the price attribute. With a price coefficient in hand you can compute the share shift it causes.

This is the value of a discrete choice model: it translates a policy variable (subsidy, licence, driving restriction) into an attribute change, and then into a share change — far cleaner than guessing from a sales time series.

Source: The example follows the module reading; the policy-as-attribute mapping is this chapter's.

Choice-Set Design: Get It Wrong and the Coefficients Are Wrong

Three things

1. Consideration set: the options a consumer would really consider, not the whole market;
2. Availability: some options cannot be bought in some regions;
3. Common denominator: the choice set must be consistent, or coefficients are not comparable.

Common mistakes

Putting twenty alternatives on one screen (respondents read the first three); or giving different people different choice sets without modelling that difference.

This chapter uses 3 alternatives $\times$ 800 occasions, three per screen — keeping the choice set to 3–6 alternatives is a well-tested rule of thumb.

Source: Choice-set design follows Louviere et al. (2000); the rule of thumb is this chapter's.

Summary: From Ratings to Choice

Method	Output	What you use it for
Conjoint	attribute utilities, importance	product configuration, pricing
Conditional logit	coefficients, shares	market simulation, policy evaluation
Nested / mixed	correlated substitution, heterogeneity	sharper share forecasts

One chain: attributes $\rightarrow$ utility $\rightarrow$ probability $\rightarrow$ share. The next three parts (hedonic, response models, attribution and CLV) are all variants of that chain on different data.

Source: Summary prepared for this chapter.

Hedonic Pricing: Break the Product into Attributes

The basic idea

A good has no single price; it is the sum of the prices of a basket of attributes: house price = area + school district + metro + age + floor.

Implicit prices

Each regression coefficient *is* the implicit price of an attribute — what ten more square metres is worth, what one kilometre closer to the metro is worth.

Housing is the most mature application of the method; the same machinery prices cars, phones, wine and art and builds quality-adjusted price indices.

Source: The hedonic method follows Rosen (1974) and Lancaster's (1966) consumer theory.

The Hedonic Specification and Semi-elasticities

The usual form

$$\ln p_i = \beta_0 + \sum_k \beta_k x_{ik} + \varepsilon_i.$$

The log is there because prices are right-skewed; β_k reads as a one-unit rise in the attribute moves price by $100(e^{\beta_k} - 1)\%$.

Two alternatives

Log–log (attributes logged too): the coefficient is the elasticity itself; semi-parametric (a spline in area): avoids getting the functional form wrong.

Measured here: area $+0.93\%$ per m^2, school district $+19.8\%$, metro -4.95% per km, age -0.71% per year, floor $+0.21\%$ per storey, with $R^2 = 0.672$ — the coefficients recover the values used to generate the data (frame 43 reproduces them).

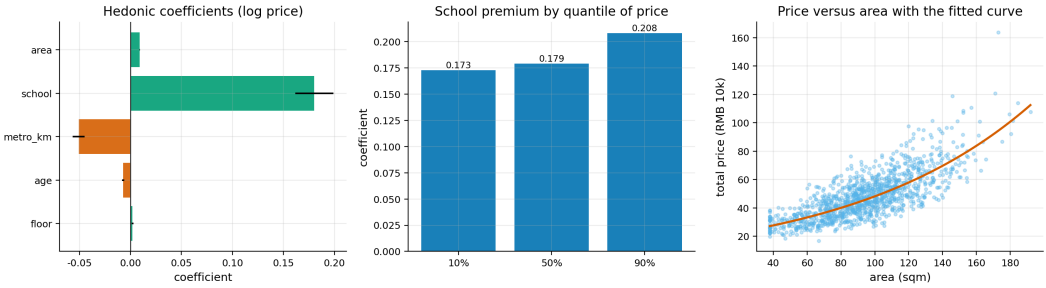
Source: The semi-log specification and its reading follow Rosen (1974) and empirical practice; the numbers come from 115_04.

[Code] A Housing Hedonic Regression with a Quantile Check

```
housing = make_housing(1200)                          # 1,200 transactions
housing["log_price"] = np.log(housing["price"])        # work in logs
model = smf.ols("log_price ~ area + school + metro_km + age + floor", data=housing).fit(cov_type="HC1") # OLS with robust SEs
coefs = pd.DataFrame({"term": model.params.index,      # coefficient names
                      "coef": model.params.values.round(4), # point estimates
                      "se": model.bse.values.round(4),    # robust standard errors
                      "p": model.pvalues.values.round(4)}) # p values
semi = (np.exp(model.params) - 1) * 100                # semi-elasticities in percent
quant = smf.quantreg("log_price ~ area + school + metro_km + age + floor", housing) # quantile regression
for q in (0.1, 0.5, 0.9):                             # three points of the distribution
    fit = quant.fit(q=q)                              # fit at that quantile
    q_rows.append({"quantile": q, "area": float(fit.params["area"]), # area effect
                   "school": float(fit.params["school"]), # school effect
                   "metro_km": float(fit.params["metro_km"])}) # metro effect
```

Measured: $R^2 = 0.672$; the school premium is +18.9% at the 10th percentile, +19.6% at the median and +23.0% at the 90th — the premium is largest on the most expensive homes, so a single average coefficient hides a real difference across price bands. Full program `115_04_hedonic.py`.

[Sample + Visualisation] How to Read Hedonic Coefficients



Left: the five coefficients with 95% intervals — school district, area and floor are positive; distance to the metro and building age are negative. Middle: how the school premium differs across price quantiles (largest on the most expensive homes). Right: price against area with the fitted curve.

Program `l15_04_hedonic.py` · coefficients `l15_hedonic_coefs.csv` · quantiles `l15_hedonic_quantiles.csv`

Three Uses of Hedonic Prices

Pricing

New-project pricing: compute the "fair" price from the attributes, then compare with competitors.

Price index

A quality-adjusted index: if prices rose 5%, how much is the same house getting dearer and how much is the house getting better?

Policy evaluation

A new metro line, a school-district change, a purchase restriction — treat the policy as an attribute change and read off the change in implicit prices.

The third use is the most often forgotten and the most valuable: it turns a policy shock into an estimable coefficient (the logic of causal inference).

Source: The three uses follow the hedonic literature and price-index practice (for example the Case–Shiller methodology).

Response Models: Marketing Spend to Sales Response

Three kinds of spend

Advertising (media exposure), promotion (discounts, coupons) and price (regular repricing). All three act on sales through the demand curve.

Three things to estimate

1. direction: does more spend sell more;
2. shape: does the marginal effect fall (saturation);
3. lag: does today's spend still work tomorrow (adstock).

The most common marketing mistake is to estimate the direction and ignore the shape — so "spend a bit more" becomes the default answer even when the marginal return is already zero or negative.

Source: The response-model framework follows Hanssens, Parsons & Schultz (2001); the three questions are this chapter's organisation.

Four Functional Forms

Form	Expression	Meaning
Linear	$S = a + bX$	every yuan works equally hard (unrealistic)
Logarithmic	$S = a + b \ln X$	diminishing, with no ceiling
Power	$S = aX^b$	diminishing when $b < 1$, constant elasticity
ADBUDG	$S = b + (a - b)\frac{X^c}{d + X^c}$	floors and ceilings, S-shaped: closest to reality

This chapter uses the Hill / ADBUDG form: $f(X) = \frac{X}{X+K}$, where K is the half-saturation point — spend K and you get half of the maximum response. That number is the one to report to management.

Source: The functional forms follow Hanssens et al. (2001) and Little's (1970) ADBUDG model.

adstock: Carryover and Decay of Advertising

The recursion

$$A_t = S_t + \lambda A_{t-1}, \qquad 0 < \lambda < 1.$$

Today's effect = today's spend + the residue of yesterday's effect.

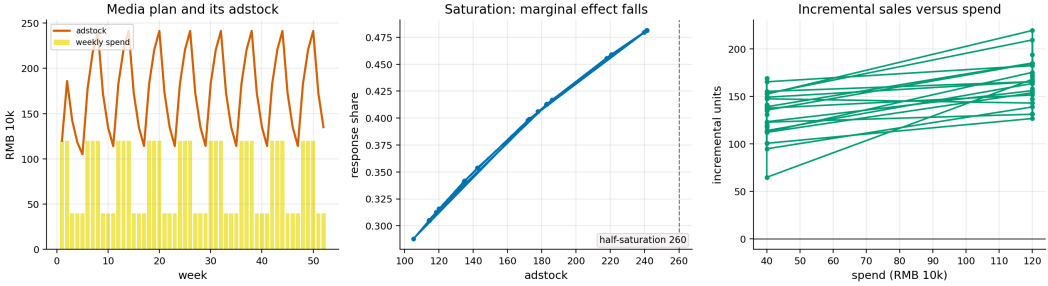
What λ means

$\lambda = 0.5$ means "half of it decays each week"; the larger λ, the longer advertising is remembered — and the easier it is to overstate the long-run effect.

Measured here: at $\lambda = 0.55$, a peak spend of 1.2m produces a peak adstock of 2.41m — so more than half of the "current effect" is residue from past spend. Without an adstock term you credit all of it to the current week.

Source: adstock follows Broadbent (1979) and Hanssens et al. (2001); the numbers come from 115_04.

[Sample + Visualisation] The Response Curve and Its Saturation Point



Left: a pulsing media plan and its adstock curve (advertising memory flattens the peaks). Middle: the saturation curve — the more you spend, the smaller the marginal response; the dashed line is the half-saturation point at 260. Right: incremental sales against spend. The rule: set spend where marginal revenue equals marginal cost, not where total response is largest.

Program `l15_04_hedonic.py` · series `l15_response_curve.csv` (52 weeks)

Promotion Response: Split Sales into Baseline and Increment

Two components

Baseline: what would have sold without the promotion; increment: the extra units the promotion brought. Only the increment is worth paying for.

Three traps

1. purchase acceleration: the increment is borrowed from the future;
2. channel switching: demand simply moved from another channel;
3. category expansion: people who would not have bought the category at all — this is real increment.

Measured here: the 12 promotion weeks average $+15.9$ units, about 51% above the regular weeks. But without a control group you cannot separate "the promotion worked" from "sales would have risen anyway" — which is what the experiment part is for.

Source: The decomposition follows Blattberg, Kim & Neslin (2008); the three traps are standard findings in marketing empirics.

What the Three Models Share: Attributes or Spend to Value

Model	Regressors	Outcome	Key parameter
Conjoint	attribute levels	utility, choice	part-worth
Hedonic	product attributes	price	semi-elasticity β_k
Response model	marketing spend	sales	half-saturation K, λ

All three break a complicated object into comparable dimensions and price each dimension. The difference is only what gets broken up: conjoint breaks the product, hedonic breaks the price, the response model breaks the spend.

Source: The unified view is this chapter's organisation.

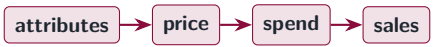
Summary: From Attribute Value to Spend Response

Three sentences

Hedonic regression prices the attributes; the response model prices the spend; adstock reminds you that effects cross periods.

Shared traps

Ignoring collinearity (attributes that move together), ignoring lag structure, and reading correlation as causation.



Next part: how the customer moves through the journey, and who should get the credit.

Source: Summary prepared for this chapter.

Clickstream: Sessions, Events and the Funnel

Three grains

Event (one click or impression) → session (split by a timeout, usually 30 minutes) → user (across sessions, which needs an ID).

Four standard metrics

Sessions, bounce rate, add-to-cart rate, conversion rate. Each has a different denominator — mix them up and the numbers are wrong.

The data here are session-level: 6,000 sessions, each with a channel path (for example "Feed ads > Search ads > Direct") and a funnel depth. That is the minimum viable dataset for attribution.

Source: The clickstream structure follows web-analytics practice (the GA4 event model); the data are in 115_05.

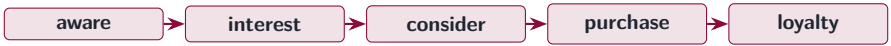
CRM and RFM: The Cheapest Customer Segmentation

Dimension	Meaning	How to bin it
R (recency)	how long since the last purchase	smaller is better
F (frequency)	how many purchases in a period	larger is better
M (monetary)	how much has been spent in total	larger is better

Three steps: bin and score (1–5) → combine into tiers (555 = high-value and active) → attach an action to each tier (the strategy matrix on frame 82). RFM needs no model and a business audience reads it instantly; the cost is that it ignores inter-purchase time and category.

Source: RFM follows database-marketing practice (Hughes 1994); the comparison with clustering is this chapter's.

The Customer Journey in Five Stages



Every stage has a metric

Impressions → click-through rate → add-to-cart rate → conversion rate → repurchase rate. The point of a funnel is to show which step leaks.

A journey is not a straight line

Real paths loop back, cross devices and touch the physical store. That is why attribution is always contested (frames 58–61).

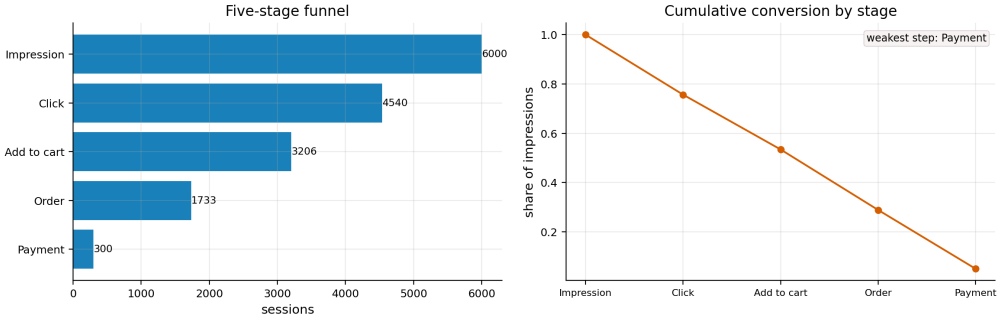
Source: The journey framework follows Lemon & Verhoef (2016); the loops and cross-device warning are this chapter's.

[Code] Funnel, Channel Performance and Attribution Splits

```
sessions = make_clickstream(6000)                       # 6,000 sessions
for k, stage in enumerate(STAGES):                      # loop over the five stages
    reached = int((sessions["depth"] >= k + 1).sum())  # sessions that got this far
    funnel.append({"stage": stage, "sessions": reached}) # the row
funnel["rate_vs_prev"] = (funnel["sessions"] / funnel["sessions"].shift(1)).round(4) # step conversion
def attribute(path, revenue, rule):                      # split one conversion across touches
    channels = [c.strip() for c in path.split(">")]     # the touch sequence
    if rule == "last":                                  # all credit to the last touch
        credit[channels[-1]] = revenue                  # single channel
    elif rule == "linear":                              # equal split
        for c in channels:                              # every touch gets the same share
            credit[c] = credit.get(c, 0.0) + revenue / n # add the share
    elif rule == "time_decay":                          # recent touches weigh more
        weights = np.array([0.5 ** (n - 1 - i) for i in range(n)]) # halve every step back
        weights = weights / weights.sum()               # normalise
for _, row in conv.iterrows():                          # loop over conversions
    for r in rules:                                     # loop over the rules
        for c, v in attribute(row["path"], row["revenue"], r).items(): # split this conversion
            credit[r][c] = credit[r].get(c, 0.0) + v    # accumulate
```

Measured: 6,000 sessions, conversion rate 5.0%; the weakest step is order → payment (0.173 survives); 300 converting sessions are split across the four rules. Full program `115_05_clickstream.py`.

[Sample + Visualisation] The Five-Stage Funnel and Its Weakest Step



Left: sessions at each stage, impression → click → add to cart → order → payment. Right: cumulative conversion. The weakest step is order to payment (0.173 survives) — usually that means friction in the payment flow, not weak marketing. Fix the leak at the bottom before buying more traffic at the top.

Program `l15_05_clickstream.py` · funnel table `l15_funnel.csv`

Attribution: Four Heuristic Rules

Rule	How it splits	The assumption it makes
Last click	all credit to the final touch	the final touch closed the sale
First click	all credit to the first touch	the first touch created awareness
Linear	equal split across touches	every touch contributes the same
Time decay	recent touches weigh more	recent influence is stronger

None of the four is "wrong", but each is biased. Last click overrates search ads (the customer searches the brand after deciding) and underrates brand advertising; first click does the reverse. So a report should always give at least two definitions.

Source: The rules follow digital-marketing practice and the review by Shao & Li (2011).

Attribution Bias: Why Last Click Is Always Overrated

Three mechanisms

1. brand-term capture: the customer already decided, then searched the brand;
2. assist: the feed ad built the memory, but the last click was a search;
3. trackability: a measurable channel (search) looks better than an unmeasurable one (offline, word of mouth).

The consequence

Budget is systematically shifted towards the channel that closes the sale, while the upper funnel that actually acquires customers is cut — so new customers shrink over time.

This is exactly why the A/B and incrementality material comes after attribution: attribution tells you how to split the credit, the experiment tells you whether the spending did anything.

Source: The three mechanisms follow attribution practice; the long-run consequence is a standard empirical finding.

Data-Driven Attribution: Shapley and Markov

Shapley value

Treat a conversion as a cooperative game: each channel is a player and the credit is the average marginal contribution. Computing it needs a value for every subset of channels: 2^k combinations.

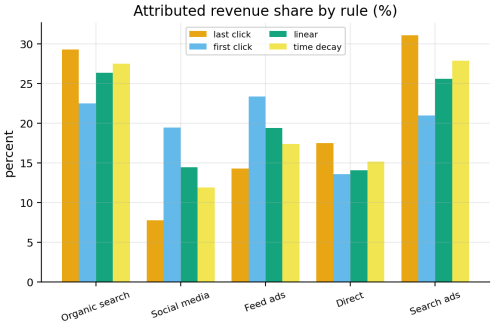
Markov chain

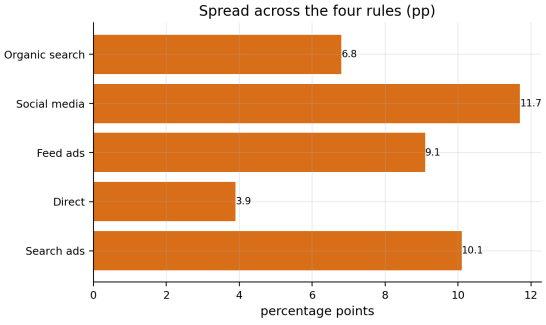
Treat states as nodes (each channel + conversion + dropout), use transition probabilities to compute how much the conversion rate falls when a channel is removed — the removal effect.

Both are better than a heuristic, but both need a lot of path data and neither proves causation — they are still a split over correlations. Causality needs an experiment or a marketing-mix model (frames 86, 88).

Source: Shapley attribution follows Shapley (1953) and Dalessandro et al. (2012); Markov attribution follows Anderl et al. (2016).

[Sample + Visualisation] One Dataset, Four Attribution Splits





Left: the revenue share of the five channels under the four rules. Right: the largest gap a channel shows across the rules, in percentage points. Social media swings from 7.8% under last click to 19.5% under first click — a 11.7-point swing, while search ads run the other way (31.1% to 21.0%). Report which definition you used, and why.

Program `l15_05_clickstream.py` · shares `l15_attribution.csv` · channels `l15_channel_rfm.csv`

Privacy Constraints: After the Cookie

Three changes

- third-party cookies are being retired, so cross-site tracking gets harder;
- IDFA and advertising IDs need explicit consent;
- walled gardens: data no longer leaves the platform (frame 63).

What it does to marketing analytics

1. attribution only works inside a platform; cross-platform needs a mix model or an experiment;
2. user-level tracking gets expensive, so first-party data appreciates;
3. reports must state the data source and the consent basis.

This is the biggest structural change in marketing analytics in 2026: observability falls, so causal inference and experiments matter more.

Source: The privacy changes follow Apple's ATT (2021), the Chrome third-party cookie retirement and the GDPR / Personal Information Protection Law.

Identity Resolution and Privacy-Enhancing Technologies

Technology	What it does	The cost
First-party graph	join your own touchpoints on phone or member ID	needs user consent
Data clean room	both sides compute jointly in a controlled space	platform lock-in
Differential privacy	publish statistics with added noise	loss of precision
Federated learning	data stays on the device, only gradients travel	engineering and communication cost

The principle: do not use a personal identifier if you can avoid it; when you must, use the minimum necessary in an auditable way. If a course project scrapes platform comments, it still has to anonymise them.

Source: The technologies follow 2026 industry practice; the principle is the minimum-necessary requirement of the Personal Information Protection Law.

Summary: Attribution Is a Split, Not a Measurement

Three sentences

The funnel finds the leak; attribution splits the credit; neither proves causation.

What the report must state

Which attribution definition was used, which channels are sensitive to it, and which data privacy constraints make unavailable.



Next part: turn preference into recommendation, and price into profit.

Source: Summary prepared for this chapter.

Recommender Systems: Problem Definition and Evaluation

The problem

Given a user's history, pick the k items from the catalogue that are most likely to be liked, and show them in order. Ranking is closer to the real objective than classification.

Metrics

precision@k (how many of the top k hit), NDCG (which accounts for position), coverage (is the catalogue being used), cold start (new users, new items).

The key warning: a good offline metric does not mean a good online result — offline measures "guessed right", online wants "clicked and bought". So a recommender always needs an A/B test (frame 84).

Source: The metrics follow Rendle et al. (2009) and recommender practice; the warning is this chapter's.

Collaborative Filtering: Similar Users, Similar Items

User-based

Find the people whose ratings look like the target user's and use their ratings to predict for the target.

Item-based

Find the items liked by the same people as the target item. Industry prefers item-based: items are more stable than users.

Both share two problems: sparsity (95% of the rating matrix is empty) and cold start. Matrix factorisation is designed for the first (frame 67); the second needs content features or an exploration policy.

Source: Collaborative filtering follows Sarwar et al. (2001); the sparsity and cold-start discussion is this chapter's.

Matrix Factorisation: Latent Factors and ALS

The model

$$\hat{r}_{ui} = u_i^\top v_u, \qquad \min_{U,V} \sum_{(u,i)\in\Omega} (r_{ui} - u_i^\top v_u)^2 + \lambda(\|U\|^2 + \|V\|^2).$$

Only observed ratings enter the objective.

How to solve it: ALS

With V fixed, each user's problem is a linear system (the code on frame 68); alternate the two solves and it converges in a few rounds. It parallelises better than SGD.

Measured here (500 users $\times$ 60 items, 24% observed): test RMSE 1.143, precision@10 0.056. Those numbers are not pretty — and that is what real recommendation looks like: the matrix is too sparse for anyone to promise 0.9.

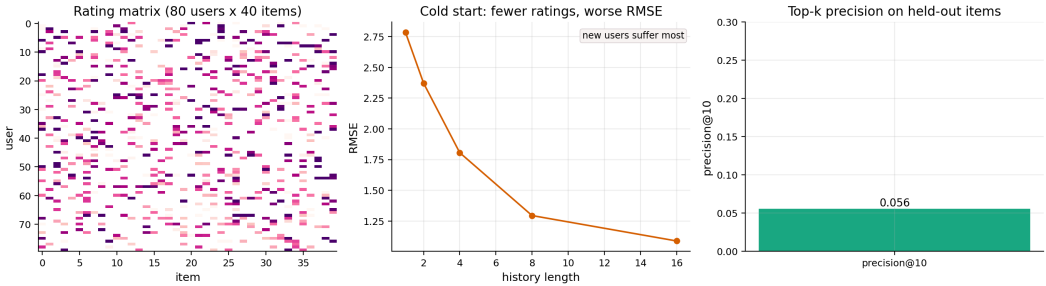
Source: Matrix factorisation follows Koren, Bell & Volinsky (2009); ALS follows Hu, Koren & Volinsky (2008).

[Code] Alternating Least Squares and precision@k

```
def als(Rmat, k=8, reg=0.08, iters=25):              # alternating least squares
    n_u, n_i = Rmat.shape                            # dimensions
    U = rng.normal(0, 0.1, (n_u, k))                 # user factors
    V = rng.normal(0, 0.1, (n_i, k))                 # item factors
    for _ in range(iters):                           # alternating updates
        for u in range(n_u):                         # fix V, solve for each user
            idx = np.where(np.isfinite(Rmat[u]))[0]  # items this user rated
            Vt = V[idx]                              # the relevant item factors
            A = Vt.T @ Vt + reg * np.eye(k)          # normal equations
            b = Vt.T @ Rmat[u, idx]                  # right-hand side
            U[u] = np.linalg.solve(A, b)             # solve
    return U, V                                      # return the factors
scores[seen] = -np.inf                               # do not recommend seen items
top = np.argsort(scores)[::-1][:k]                   # the top-k ranking
hits += len(set(top) & set(test_items))               # hits among the top-k
```

Measured: RMSE 1.143, precision@10 0.056; and cold start — as history shrinks from 16 ratings to one, RMSE worsens from 1.09 to 2.79 (figure on frame 69). Full program `115_06_recommender.py`.

[Sample + Visualisation] Sparsity, Accuracy and Cold Start



Left: the rating matrix (bright cells are observed; the large blank areas are the sparsity). Middle: the cold-start curve — less history, worse RMSE. Right: top-10 precision. Conclusion: the bottleneck in recommendation is missing data, not a model that is not deep enough.

Program `l15_06_recommender.py` · ratings `l15_ratings.csv` · cold start `l15_cold_start.csv`

Deep and Sequential Recommenders

Two towers

One tower encodes the user, one the item, and the score is their inner product; serving can then use vector search.

Feature crossing

Cross "user attribute $\times$ category $\times$ time" explicitly (for example DeepFM); in effect the model learns its own interaction terms.

Sequence models

Model the behaviour sequence with an RNN or a transformer; used for purchase prediction and session-based recommendation.

The engineering order: collaborative filtering $\rightarrow$ matrix factorisation $\rightarrow$ deep models. Get the cheapest baseline first, then ask how much the deep model adds — in most settings the gain is 5–15% while the engineering effort doubles.

Source: Two towers and DeepFM follow Covington et al. (2016) and Guo et al. (2017); the engineering order is this chapter's.

Price Elasticity: How Much Does a 1% Rise Cost You

Definition and estimation

$$\varepsilon = \frac{\partial \ln q}{\partial \ln p},$$

estimated with a log–log regression: $\ln q = a + \varepsilon \ln p + \text{controls}$.

Reading it

$|\varepsilon| > 1$ is elastic (a price cut raises revenue); $|\varepsilon| < 1$ is inelastic (a price cut lowers revenue).

Measured here: $\varepsilon = -1.842$ against a true -1.85, a promotion effect of $+0.296$. Once the elasticity is estimated, the optimal price is one step of algebra (frame 72).

Source: The definition and estimation follow standard demand analysis; the numbers come from 115_07.

[Code] Log-Log Elasticity and Profit-Maximising Pricing

```python
model = smf.ols("log_units ~ log_price + promo", data=sales).fit(cov_type="HC1") # robust standard errors
elasticity = float(model.params["log_price"])          # the own-price elasticity
def demand(price):                                     # expected units at a price
    return float(np.exp(intercept + elasticity * np.log(price) + 0.35 * promo_mean)) # log-linear demand
def profit(price):                                     # profit at a price
    return (price - unit_cost) * demand(price)          # margin times units
grid = np.linspace(20, 70, 400)                        # a price grid
profits = np.array([profit(p) for p in grid])          # profit at every price
best = float(grid[np.argmax(profits)])                 # the profit-maximising price
markup_rule = unit_cost * elasticity / (1 + elasticity) # the Lerner price for log-linear demand
sales["baseline"] = np.exp(intercept + elasticity * np.log(sales["price"])) # no-promotion baseline
sales["increment"] = sales["units"] - sales["baseline"] # incremental units
promo_weeks = sales[sales["promo"] == 1]               # promotion weeks only
```

Measured: $\varepsilon = -1.842$, promotion coefficient 0.296; the optimal price is 37.17, against 37.20 from the Lerner formula — the two routes confirm each other. Full program `115_07_pricing.py`.

Promotion: Discounts, Coupons and Thresholds

Three instruments

A straight discount (simple, but damages the price image), coupons (targetable and redeemable), spend thresholds (raise the basket size).

The psychology of a threshold

"Spend 199, save 30" pushes someone who was going to spend 150 up to 199 — 30 of margin buys 49 of extra revenue, but it also trains customers to wait for promotions.

Evaluating a promotion means answering where the increment came from: new customers (real increment), existing customers buying early (time shifting), orders moved from another channel (channel shifting), or a change in category choice (category expansion). Only the first is worth funding in the long run.

Source: The instruments and threshold effects follow Blattberg et al. (2008); the four-way split is this chapter's organisation.

[Sample + Visualisation] Demand Curve, Optimal Price and Promotion Lift



Left: the demand curve (red points are promotion weeks). Middle: profit rises and then falls with price; the optimum is 37.17 while the current price of 39 sits slightly above it — but the profit curve is flat there, so the gain is small. Right: sales split into baseline plus promotion increment. A promotion looks like "selling more", while the profit optimum is somewhere else.

Program `115_07_pricing.py` · weekly data `115_price_sales.csv` · decomposition `115_promotion_decomposition.csv`

Price Discrimination: Ethics and Compliance

Three degrees

First degree: a price per person (what people call "algorithmic gouging"); second degree: price falls with quantity; third degree: price by group (student, member).

The compliance line

The Price Law and the Personal Information Protection Law require clearly marked prices and no price fraud; differential pricing driven by personal characteristics carries the highest risk.

Practical advice: keep personalised pricing inside public, explainable rules (membership tiers, volume discounts) rather than an opaque personal algorithm. A report should say what the pricing rule was based on.

Source: The three degrees follow Pigou (1920); the compliance points follow the PRC Price Law and Personal Information Protection Law.

Summary: Recommendation Sells More, Pricing Sells Dearer

Two different objectives

Recommendation optimises conversion and clicks (sell more items); pricing optimises unit margin (sell at a higher price).

Shared constraints

Both need data (behaviour, transactions), both need an experiment, and both need compliance.



Next part: who leaves, what they are worth, and how an experiment proves it.

Source: Summary prepared for this chapter.

Churn Prediction: Definition, Window and Label

Three things to settle first

1. What counts as churn: cancelling, 90 days inactive, downgrading?
2. Observation and prediction window: features from the last 6 months, prediction over the next 3;
3. Who is in the sample: only active users, or dormant ones too.

Class imbalance

Churn rates usually run 5–15%. Use AUC or the PR curve rather than accuracy, and set the threshold from expected loss (frame 78).

The most common methodological error: building features from information available only after the churn ("the last complaint before cancelling"). It inflates AUC and collapses in production.

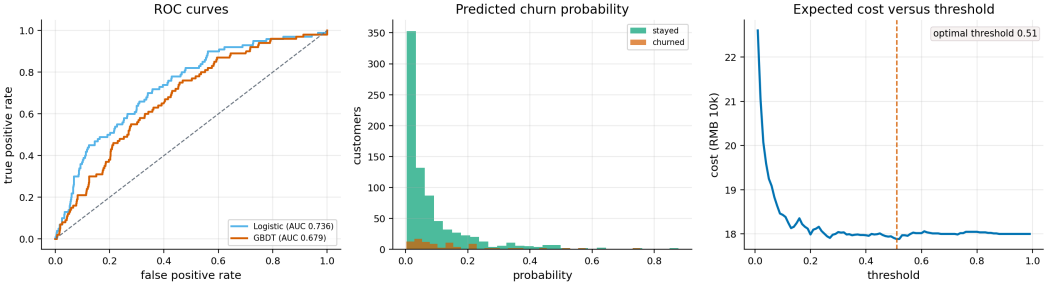
Source: The setup and the traps follow subscription-business practice (for example the retention models of Fader & Hardie).

[Code] Churn Models and the Expected-Loss Threshold

```
logit = LogisticRegression(max_iter=2000).fit(X_tr, y_tr) # logistic regression
gbm = GradientBoostingClassifier(random_state=42, n_estimators=250, max_depth=3).fit(X_tr, y_tr) # gradient boosting
p_logit = logit.predict_proba(X_te)[:, 1]              # predicted probabilities
p_gbm = gbm.predict_proba(X_te)[:, 1]                  # predicted probabilities
for t in thresholds:                                   # loop over thresholds
    flagged = p_gbm >= t                               # who gets contacted
    n_flagged = int(flagged.sum())                     # how many
    caught = int(((y_te == 1) & flagged).sum())        # churners who are flagged
    missed = int(((y_te == 1) & ~flagged).sum())       # churners who are missed
    cost = (n_flagged * cost_contact                   # contact cost
          + missed * value_at_risk                     # lost revenue from misses
          + caught * (1 - success_rate) * value_at_risk) # non-saved customers still leave
best = min(curve, key=lambda r: r["cost"])             # the cheapest threshold
```

Measured: logistic AUC 0.736, gradient boosting 0.679 (the signal is close to linear, so the trees overfit); the minimum expected cost sits at threshold 0.51, but the curve is flat — from 0.2 to 0.9 the cost moves by under 1%. Full program `115_08_churn.py`.

[Sample + Visualisation] Churn Probability, ROC and the Cost Curve



Left: the two ROC curves (AUC 0.736 against 0.679). Middle: the predicted probabilities of stayers and churners — they overlap heavily, so a perfect split is impossible. Right: expected cost against the threshold; it bottoms out at 0.51 but is almost flat. What is worth arguing about is the level of the curve (the loss ratio and the contact cost), not a magic threshold.

Program `l15_08_churn.py` · metrics `l15_churn_metrics.csv` · thresholds `l15_churn_threshold.csv`

Customer Lifetime Value: Three Methods

Method	How it is computed	Where it fits
Historical	cumulative contribution to date	mature customers, simplest
Rule of thumb	$CLV = m \cdot \frac{r}{1+i-r}$	assumes a constant retention rate
Probabilistic	fit a retention curve, then discount (frame 81)	new customers, retention that changes over time

Read the rule-of-thumb formula as monthly margin m, monthly retention r, monthly discount rate i: at $r = 0.95$ and $i = 0.01$ the multiplier is about 16 months. It assumes retention never changes — older customers are usually more loyal, so the rule understates long-run value.

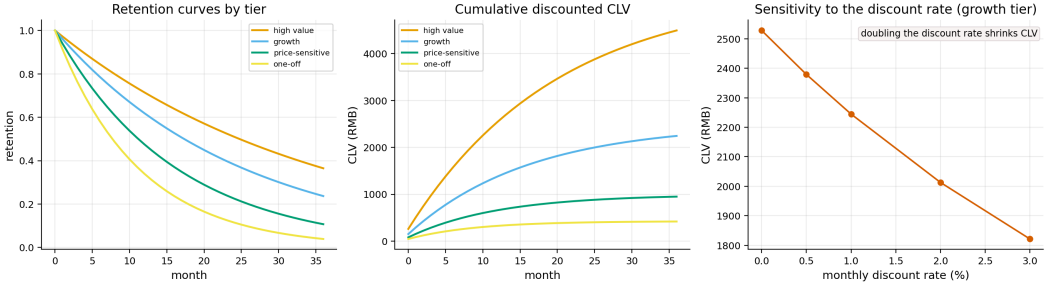
Source: The three methods follow Gupta et al. (2006) and the BG/NBD framework of Fader & Hardie (2009).

[Code] CLV: Retention Curve, Discounting and Tiers

```python
tiers = {"high value": {"m0": 260.0, "decay": 0.028, "n": 250}, # high value, slow decay
         "growth": {"m0": 150.0, "decay": 0.040, "n": 450}, # mid value
         "price-sensitive": {"m0": 80.0, "decay": 0.062, "n": 550}, # fast decay
         "one-off": {"m0": 45.0, "decay": 0.090, "n": 750}} # very fast decay
months = np.arange(0, 37)                                  # three years of months
discount = 0.01                                            # 1% monthly discount rate
for name, cfg in tiers.items():                            # loop over the tiers
    survival = np.exp(-cfg["decay"] * months)               # exponential retention curve
    margin = cfg["m0"] * np.exp(-0.012 * months)           # slowly shrinking margin
    disc = (1 + discount) ** (-months)                     # discount factors
    clv_cum = np.cumsum(survival * margin * disc)          # cumulative discounted value
summary = clv_long.groupby("tier")["clv_cum"].max().round(1) # three-year CLV by tier
weighted = float((summary * weights).sum() / weights.sum()) # customer-weighted CLV
for r in rates:                                            # loop over the discount rates
    total = float(np.sum(survival * margin * (1 + r) ** (-months))) # discounted sum
```

Measured: 36-month CLV of 4,496 for high value, 2,245 growth, 949 price-sensitive and 418 one-off; customer-weighted CLV 1,485. Raising the discount rate from 0% to 3% a month cuts CLV from 2,528 to 1,822 — about 28%. Full program `115_09_clv.py`.

[Sample + Visualisation] Retention Curves and CLV



Left: the retention curves of the four tiers — high-value customers stay longest. Middle: cumulative discounted CLV, still rising after three years. Right: sensitivity to the discount rate. The same customers are worth 28% less once the discount rate goes from 0 to 3% a month — so the assumption has to be stated in the report.

Program `115_09_clv.py` · curves `115_clv_curve.csv` · sensitivity `115_clv_sensitivity.csv`

Value × Churn Probability: The Four-Quadrant Strategy Matrix

The four quadrants

- high value + low churn: maintain (do not disturb);
- high value + high churn: priority retention;
- low value + low churn: low-cost nurture;
- low value + high churn: let go, or self-serve.

Why two dimensions

Churn probability alone spends the money on low-value customers; CLV alone ignores the valuable customer who is about to leave. Only the cross-tab gives an actionable policy.

Measured here: high value (CLV 4,496, annual churn 28.5%), growth (2,245 / 38.1%), price-sensitive (949 / 52.5%), one-off (418 / 66.0%) — the higher the value, the lower the churn, so "priority retention" falls on the growth tier rather than the top one.

Source: The matrix is this chapter's organisation; the numbers come from `115_09` (synthetic data).

A/B Experiments: The Design Decides Whether You Can Conclude Anything

Five design decisions

1. randomisation unit: user, session, region?
2. sample size: compute power first (frame 85);
3. metrics: one primary, several guardrails;
4. duration: at least one full week (weekend effects);
5. analysis plan: write it before you run.

Three fatal mistakes

Peeking mid-experiment inflates false positives; multiple comparisons left uncorrected; sample-ratio mismatch never checked.

The data here: 5,000 users per arm, a 5.0% baseline conversion and a true lift of 0.6 percentage points. Realistic, and very small — which is exactly what makes it a good demonstration of why power matters.

Source: The design points and the mistakes follow Kohavi et al. (2020); the data are in 115_10.

[Code] Two-Proportion Test, Power Curve and CUPED

```
p_c, p_t = x_c / n_c, x_t / n_t                          # the two rates
p_pool = (x_c + x_t) / (n_c + n_t)                       # pooled rate under H0
se_pool = np.sqrt(p_pool * (1 - p_pool) * (1 / n_c + 1 / n_t)) # standard error under H0
z = (p_t - p_c) / se_pool                                # the z statistic
p_value = 2 * (1 - stats.norm.cdf(abs(z)))               # two-sided p value
lift = (p_t - p_c) / p_c                                 # relative lift
def power_two_prop(p1, p2, n):                           # analytic power of a two-proportion test
    se0 = np.sqrt(2 * p_bar * (1 - p_bar) / n)           # standard error under H0
    se1 = np.sqrt(p1 * (1 - p1) / n + p2 * (1 - p2) / n) # standard error under H1
    return float(stats.norm.cdf((abs(p2 - p1) - crit * se0) / se1)) # power
theta = float(np.cov(post, pre)[0, 1] / np.var(pre))     # the CUPED coefficient
post_adj = post - theta * (pre - pre.mean())             # the adjusted outcome
```

Measured: control 4.78%, treatment 5.78%, relative lift 20.9%; $z = 2.24$, $p = 0.0254$, 95% interval on the difference [0.0012, 0.0188] (just clear of zero). CUPED cuts the variance by 13.0%. Full program `115_10_experiment.py`.

Incrementality: Why "Lift" Is Not "Increment"

Three confounders

1. selection bias: people who see the ad were already more likely to buy;
2. spillover: the control group is affected (same building, same social circle);
3. network effects: on a marketplace, one side moves the other.

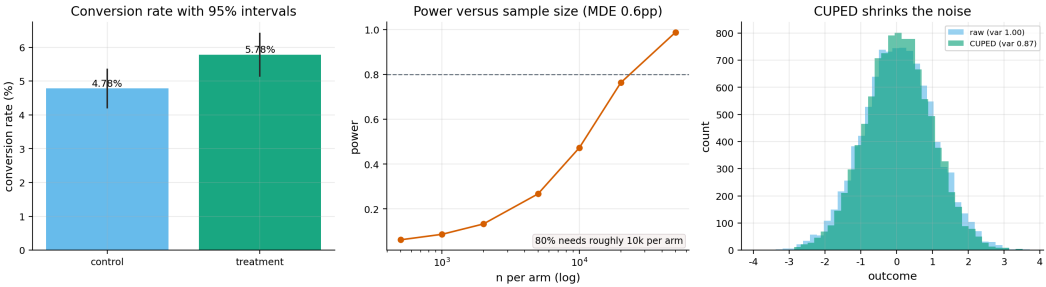
How to measure increment

PSA / ghost ads (show the control group a blank ad), geo experiments (randomise by region), uplift models (estimate the individual treatment effect).

In one sentence: an A/B test asks whether there was an effect, incrementality asks how much of it was genuinely new. The gap is largest for big budgets and brand advertising.

Source: Incrementality experiments and uplift modelling follow Lewis & Rao (2015) and marketing practice; spillover and network effects follow Kohavi et al. (2020).

[Sample + Visualisation] The Power Curve and the Minimum Detectable Effect



Left: the two conversion rates with 95% intervals. Middle: power rises with sample size; to reach 80% at a 0.6-point lift you need about 22,000 per arm. Right: CUPED shrinks the variance. At 5,000 per arm the power is only about 27% — which is why so many "insignificant" experiments are really underpowered.

Program `l15_10_experiment.py` · power table `l15_power_curve.csv` · result `l15_ab_result.csv`

Marketing-Mix Models and Causality

The MMM trio

adstock (the lag) + saturation (the non-linearity) + controls (season, price, competitors); a regression then decomposes each channel's contribution.

Its limits

Correlation $\neq$ causation (channel budgets move together); it works at weekly or monthly grain; collinearity is severe.

The 2026 mainstream is to use all three together: the mix model allocates the macro budget, experiments establish single-channel causality, attribution drives day-to-day optimisation. They calibrate each other rather than replace each other.

Source: MMM follows Hanssens et al. (2001) and 2026 platform practice; the division of labour is this chapter's.

Four Things That Changed in 2026

Generative content

Copy, images and video at scale; evaluation moves from "click-through" to "brand safety and factual consistency".

Brand safety

What content the ad sits next to; needs content review and exclusion lists.

Conversational commerce

Browsing and payment inside a chat; the funnel compresses into one conversation.

Multimodal analytics

Understanding image, video and live-stream content, for creative testing and reputation monitoring.

What the four share: content has moved from "made by people" to "generated by people and screened by machines", so measurement shifts from "how many impressions" to "is the content right, is it safe, and did it add anything".

Source: 2026 marketing-technology practice (generative content, brand safety, conversational commerce, multimodal analytics), organised for this chapter.

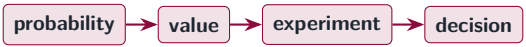
Summary: From Prediction to Decision

Three sentences

Churn has to be converted into money before a threshold is set; CLV has to be discounted; an A/B test needs a power calculation first.

The shared discipline

Every claimed effect answers three questions: who was the control group, where did the increment come from, and which definition was used.



The last part: the tools, the glossary, and a one-page recap.

Source: Summary prepared for this chapter.

Tool Stack

Tool	Strength	Where it is used here
Qualtrics / Wenjuanxing	survey fielding and quotas	frames 9–18
pandas	cleaning and reshaping	every program
statsmodels	OLS, quantile regression, logit	frames 27, 33, 43, 72
scikit-learn	classification, clustering, metrics, PCA	frames 16, 78
scipy	optimisation, distributions, tests	frames 33, 85
Experiment platform	assignment, metrics, SRM checks	frames 84–87

This chapter deliberately adds no new dependency: even the matrix factorisation is a hand-written ALS in numpy, so that you can see what the model is doing rather than which library was called.

Source: The division of labour follows each tool's documentation and this chapter's practice.

Marketing Data and Compliance Checklist

Six items that must be written down

1. where the data came from and how consent was obtained;
2. when it was collected and what it covers;
3. how personal information was handled (anonymised, minimised);
4. retention period and access control;
5. cross-border transfer, if any;
6. what the model may and may not be used for.

Three common violations

- scraping platform comments without authorisation;
- using a phone number as a user ID without anonymising it;
- differential pricing on personal traits with undisclosed rules.

These six items map directly onto the "data acquisition and compliance" row of the marking rubric. If you cannot write it down, you did not do it — the marker can only judge what is written.

Source: The checklist follows the Personal Information Protection Law and marketing data practice; the violation examples are this chapter's.

Ten Common Mistakes

1. a three-item scale with no reliability check;
2. asking "which attribute is important" and calling it conjoint;
3. collinear conjoint attributes, so the utilities cannot be separated;
4. turning ratings into choice probabilities and reporting shares;
5. a hedonic regression without logs, with coefficients read as percentages;
6. reporting last-click attribution only;
7. CLV without discounting or a retention curve;
8. peeking mid-experiment, or no correction for multiple comparisons;
9. treating "lift" as "increment";
10. pricing on personal traits without disclosing the rule.

Self-check: run through these ten before submitting anything — they explain almost every marketing analysis that "looks great and collapses in production".

Source: The list is this chapter's, mostly drawn from common referee and project-review comments.

Links to Projects and Assessment

Project	Frames it uses	What it delivers
Project 2	21–29 (conjoint), 71–74 (elasticity)	attribute table + elasticity estimate
Project 3	46–52 (response), 84–87 (experiments)	response curve + experiment plan
Project 4	62–63 (privacy), 75 (pricing compliance)	compliance checklist + anonymisation plan

Against the rubric: problem definition is frames 7 and 9; method and implementation is the conjoint, logit and CLV estimation; results and validation is the A/B, power and incrementality material (frames 84–87); data acquisition and compliance is frames 62–63 and 92.

Source: The mapping follows the course outline and the project brief.

Glossary A–L

A/B test — randomised two-arm comparison
adstock — carryover of past spend
ALS — alternating least squares
attribute — a comparable product dimension
attribute importance — range share of the utilities
attribution — splitting credit across touches
baseline sales — sales without any marketing
Cronbach's α — internal consistency of a scale
carryover rate — share of the effect that persists
choice set — options really considered
churn — a customer stops buying
clickstream — the sequence of page events
cold start — a new user or a new item
collaborative filtering — similar users or similar items
common method bias — bias coming from one method
conjoint analysis — value the bundle, not the attribute
construct — a concept measured indirectly
content validity — items cover every dimension
CUPED — variance reduction with pre-period data
convergent validity — two scales of one construct agree
conversion rate — share of sessions that act
cross-price elasticity — response to a rival's price
customer journey — aware to loyal, in stages
CLV — discounted future margin
D-optimal design — information-maximising design
data-driven attribution — Shapley or Markov split
discount rate — the time value of money
discrete choice model — choosing among finite options
discriminant validity — different constructs stay apart
expected loss — cost-weighted decision rule
EFA — exploratory factor analysis
first-party data — data you collect yourself
fractional factorial — a subset of the full design
funnel — drop-off from stage to stage
hedonic pricing — price the attributes
identity resolution — join touchpoints to one user
increment — the sales actually caused
incrementality test — causal test of the lift
IIA — proportional substitution
Likert scale — agreement on a 1–5 scale
last-click attribution — credit to the final touch
level — a value of an attribute
linear attribution — equal split across touches
logit model — logistic choice probability

44 terms here, 41 on the next slide; 85 in total. The full bilingual table is `115_glossary.csv`.

Glossary M–Z

Markov attribution — the removal effect
marginal cost — cost of one more unit
marginal effect — change in probability per unit
MMM — marketing mix model
markup pricing — cost-plus price
matrix factorisation — latent user and item factors
measurement error — observed minus true value
MDE — minimum detectable effect
mixed logit — random coefficients
mode effect — the mode changes the answers
nested logit — correlation within nests
non-response bias — responders differ from the rest
NDCG — position-weighted ranking metric
operationalisation — turning a construct into items
orthogonal design — uncorrelated attribute levels
precision@k — hits among the top k
price discrimination — different prices by segment
price elasticity — percent change in quantity
PET — privacy-enhancing technology
probit model — normal-error choice model
promotion uplift — extra sales in promotion weeks
quantile regression — effects along the distribution
quota sampling — fill quotas to a profile
random utility — utility plus a random shock
RFM — recency, frequency, monetary
reliability — consistency of a measurement
response model — spend mapped to sales
retention curve — survival over time
Shapley value — average marginal contribution
sampling frame — the list you sample from
saturation — diminishing marginal response
semi-elasticity — percent change per unit
session — one continuous visit
statistical power — chance of finding a true effect
stratified sampling — draw within strata, then pool
time-decay attribution — recent touches weigh more
two-tower model — separate user and item encoders
uplift model — individual treatment effect
utility (part-worth) — marginal value of a level
validity — measuring the right thing
walled garden — closed platform data

41 terms here, 44 on the previous slide; 85 in total. Bold terms are defined where first used.

References and Further Reading

Method classics

- Green & Rao (1971); Green & Srinivasan (1990): conjoint analysis
- McFadden (1974); Train (2009): discrete choice
- Rosen (1974): hedonic pricing
- Hanssens et al. (2001): marketing response models

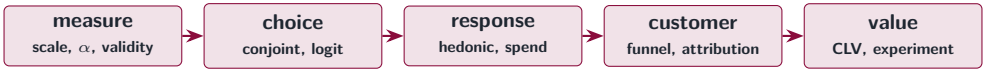
Customers and experiments

- Gupta et al. (2006): a CLV review
- Fader & Hardie (2009): BG/NBD
- Kohavi et al. (2020): trustworthy online experiments
- Lewis & Rao (2015): incrementality experiments

Reading advice: read the assumptions of a method before deciding to use it — the number is only as good as the assumption behind it.

Source: References are the original papers; the reading advice is this chapter's.

One-Page Recap



Step	Key number	In one sentence
Measurement	α 0.72–0.81	if the measurement is wrong, everything after it is wasted
Choice	price importance 35.8%	attribute value only appears when something is traded away
Response	half-saturation 260	spending has a ceiling
Customer	weakest step 0.173	fix the bottom of the funnel first
Value	CLV 1,485	discount it, and split it into tiers
Experiment	$p = 0.025$, power 27%	small effects need large samples

Source: Every number in the table comes from the ten programs of this chapter (synthetic data).

Ten Self-Check Questions

1. How many items does your scale have, and what is α?
2. Are your conjoint attributes collinear?
3. Was the importance you report estimated from ratings or from choices?
4. How do you turn the hedonic coefficient into a percentage?
5. Which attribution definition did you use, and why?
6. Where do your offline recommender metrics part from the online objective?
7. Is your elasticity estimated from cross-section or time series?
8. Is your churn threshold set by accuracy or by cost?
9. What window and what discount rate does your CLV use?
10. Is your experiment powered, and what is the MDE?

These ten map onto the ten common mistakes — answer eight of them and the report is already above average.

Source: The checklist is this chapter's.

Wrap-up: Attributes, Choice, Value

Three sentences

1. the core of marketing analytics is turning people into estimable variables;
2. every method carries explicit assumptions; if they fail, the number means nothing;
3. effects have to be proved by experiment, not narrated by attribution.

Self-check

- Can you design a trustworthy scale?
- Can you estimate and interpret attribute utilities?
- Can you compute one customer's CLV?
- Can you design a powered A/B test?

Thank you. Questions and project discussions: `Xieer.dai@szu.edu.cn`; the code, data and glossary are in `website/code/lecture15/`.

Source: Wrap-up prepared for this chapter.